

For both the memoized and dynamic programming versions, describe a scenario in which these would exhibit the worst-case time complexity and explain why this is the worst-case. Note that we are not referring to removing all seams but one. Rather, we are asking *what kind of input* (image) would cause the worst-case behavior. You do not need to do a full-blown big-O analysis.

Currently, in both memoization and dynamic programming, I *have* to visit every pixel in the current image and calculate the best seam down from each pixel per seam removal.

So, to find out the worst-case input, I analyzed what kind of image would cause a bigger time cost for calculating the best seam down from each pixel.

For dynamic programming, the cost of calculating the best seam down from a single pixel is linear in the number of pixels wide the row below it is. Specifically, I use `option-get` to get the energies of the below seams to calculate the best seam downwards from the current pixel I am on. `Option-get` is linear in the number of elements in the list it is passed, **so for wider images** (where the width is larger than the height), it will take longer for my dynamic programming version to calculate the best seam down from a certain pixel.

Similarly, in my memoization function, the cost of calculating the best seam down from a single pixel recursively asks for the best seam down from the pixels below it and uses that to calculate the best seam down from itself. For this, I also use `option-get` to access the correct below, diagonal right, and diagonal left pixels from the current pixel, and this operation is linear in the number of elements in the row below the current pixel. Thus, for wider images, (where the width is larger than the height), this would be the worst-case scenario for my function.